MAKÜ
BURDUR MEHMET AKİF ERSOY ÜNİVERSİTESİ

VERİ YAPILARI VE ALGORİTMALAR

HAFTA 10
HASHING (KARMA)

Dr. Öğr. Üyesi Ebru AYDOĞAN

Bucak Bilgisayar ve Bilişim Fakültesi

Ders İçeriği: Hafta 10

- Hashing
- Hash Fonksiyonları
- Çakışma
- Çakışmanın Giderilmesi
- Örnek Uygulamalar

Hashing (Karma) nedir?

- Hashing / Karmalama / Hesaba Dayalı Adresleme
- Hashing, herhangi bir boyuttaki verinin sabit boyutta bir çıktıya (hash değeri) dönüştürülmesi işlemidir. Bu çıktı, orijinal veri hakkında bilgi vermekle birlikte, verinin kendisini geri elde etmek mümkün değildir. Bu özelliği sayesinde hashing, birçok alanda kullanılır.

Hashing(Karma) Kullanım Alanları

- **Şifreleme:** Parolalar, hash edilerek veritabanında saklanır. Bu sayede, şifre çalınmış olsa bile, orijinal şifreye ulaşmak çok daha zor hale gelir.
- **Veri bütünlüğü:** Bir dosyanın hash değeri hesaplanarak, dosyanın daha sonraki bir zamanda değiştirilip değiştirilmediği kontrol edilebilir.
- **Veri tabanı indeksleme:** Veri tabanlarında hızlı arama yapmak için hashing kullanılır.
- **Blok zincir teknolojisi (Blockchain):** Bitcoin gibi kripto para birimlerinde, işlemler hash algoritmaları kullanılarak doğrulanır ve bir bloğa eklenir.

Hashing (Karma)

- Hashing, elimizdeki veriyi kullanarak o veriden mümkün olduğunca benzersiz bir tamsayı elde etme işlemidir.
- Bu elde edilen tamsayı, dizide tutulan verilerin indisi gibi kullanılarak verilere tek seferde erişmeyi sağlar.
- Arama metotlarında temel işlem anahtarları karşılaştırmaktır. Bir anahtarın tablo içerisinde bulunduğu pozisyona ulaşıncaya kadar arama işlemine devam edilir.
- Hash fonksiyonuyla aranan anahtar elemana doğrudan erişilebilmektedir. Hash fonksiyonu bir anahtar bilgisinin tabloda bulunduğu indeksi hesaplamaktadır.
  - Open hashing (Açık karma): Potansiyel olarak limitsiz alan kullanır.
  - Closed hashing (Kapalı karma): Bilgi kaydı için sabit alan kullanır.

Hashing (Karma)

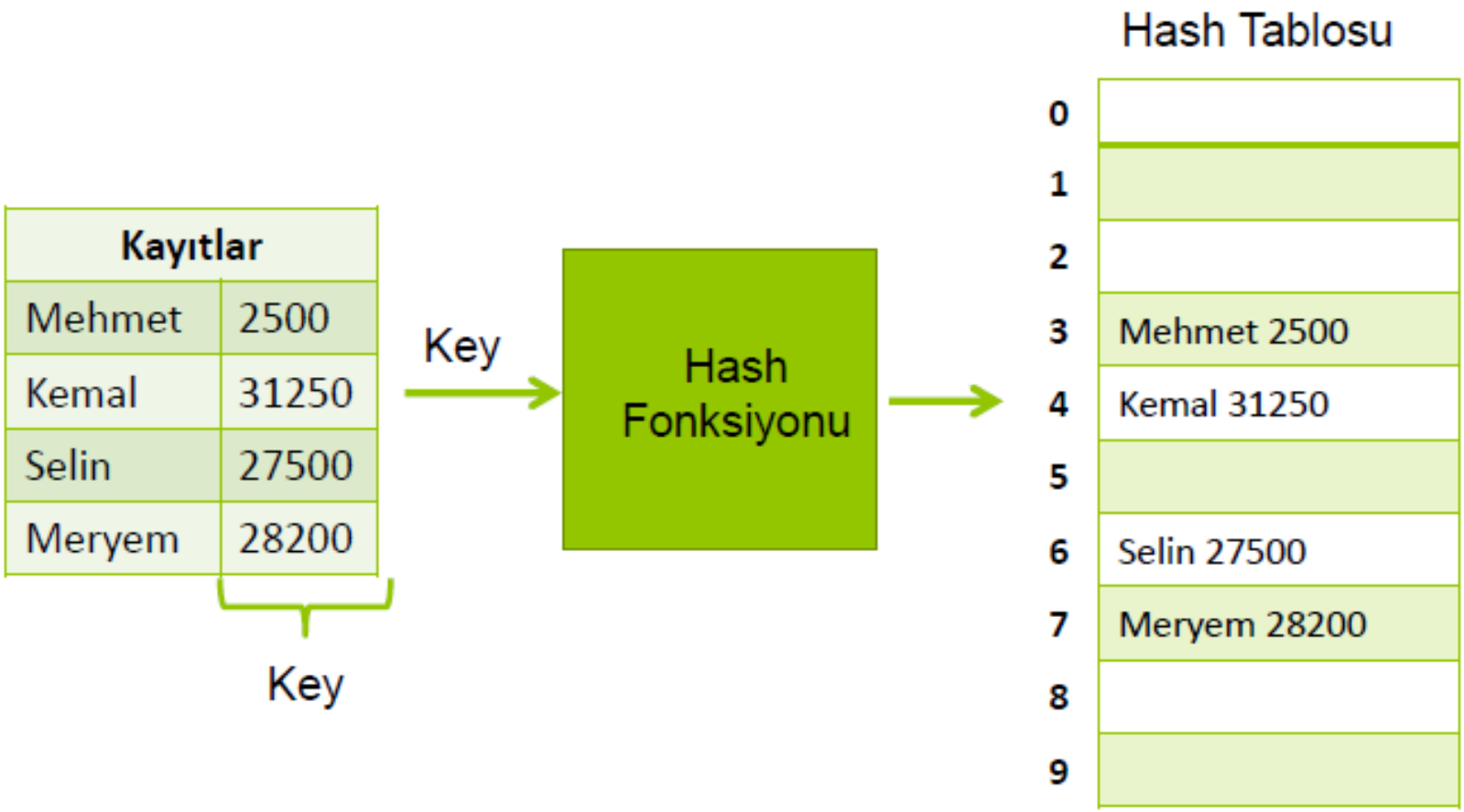
- Boyutu N olan bir tabloda, hash fonksiyonu(h(x)) bir x anahtarını, 0 ile N-1 arasında bir değerle eşleştirir.
  - h(x) = x % N
  - ***x: anahtar, N: tablo boyutu***
- Örnek:
  - 15 elemanlı bir tablo için h(x) = x % 15 (% - modlu bölüm) olarak belirlenebilir:

x	25	129	35	2501	47	36
h(x)	10	9	5	11	2	6

  - Anahtarların tablo içerisindeki yerleri ise aşağıdaki gibidir:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
-	-	47	-	-	35	36	-	-	129	25	2501	-	-	-

Hashing (Karma)



Hashing (Karma)

- MD5, SHA1 yaygın kullanılan hash algoritmalarıdır.

- Örnek:

Şifrelemek istediğiniz kelime:

merhaba

Şifrele

MD5 Şifreniz:

c39436ee452e641cde2eb992ab397911

SHA1 Şifreniz:

8af22540a3ae8c4a05e7eabb53d26cfc3393e91c

Şifrelemek istediğiniz kelime:

merhabA

Şifrele

MD5 Şifreniz:

6ecc6172f13dd50b8fb19de1aca453cf

SHA1 Şifreniz:

1a3466e72edee110f8636eed9563104fdd804de6

Hash Fonksiyonları

- Hash fonksiyonları, bir anahtarın tabloda bulunduğu indeks sırasını verir.
- Perfect hash fonksiyonu:
  - Her bir anahtara sadece bir pozisyonu eşleştiren fonksiyona denir.
- Simple perfect hash fonksiyonu:
  - Tablo boyutu ile toplam anahtar sayısı eşit olduğunda (tabloda boş yer yoksa) her bir anahtara sadece bir pozisyonu eşleştiren fonksiyona denir.
- İyi bir hash fonksiyonu nasıl olmalıdır?
  - kolay ve hızlı hesaplanabilir olmalıdır.
  - tablodaki her bir pozisyon için sadece bir anahtar atamalıdır.

Hash Fonksiyonları

- Hash fonksiyonları integer sayılarla işlem yaparlar.
- Integer olmayan anahtarlarda integer değere dönüştürme işlemi yapılır.
- Örneğin kişilere ait sağlık numarası 9635-8904 şeklinde ise aradaki tire işareti kaldırılarak 96358904 olarak alınır.
- Eğer anahtar karakterlerden oluşuyorsa karakterlerin ASCII kodları kullanılır.

Hash Fonksiyonları

Selecting Digits (Basamak Seçme)

- Anahtar üzerindeki belirlenmiş bazı haneleri seçip birleştirerek tablodaki pozisyon bulunur.
- 2. ve 5. hanelerin seçimiyle oluşturulan değer aşağıdaki gibidir:

  h(033475678) = 37

  h(023455678) = 25
- Artıları ve Eksileri
  - Yapısı basittir.
  - Anahtarları tablonun tamamına düzgün bir şekilde dağıtamaz.
  - Çakışma çok sık olabilir.

Hash Fonksiyonları

➢ Folding (Katlama)

- Anahtar birkaç parçaya bölünür ve bu parçalar kendi arasında toplanarak tablodaki pozisyon bulunur.
- Shift folding metodunda anahtarın her bir parçası değiştirilmeden tablo boyutuna göre mod ile toplanır.
  - Örnek: SSN = 123-45-6789 olarak verilsin. SSN numarası 123, 456, 789 olarak üç parçaya ayrılıp toplandığında 123+456+789 = 1368 olarak pozisyon numarası elde edilir.
- Boundary folding metodunda anahtarın parçalarının sırası değiştirilerek tablo boyutuna göre mod ile toplanır.
  - Örnek: SSN numarası 123, 456, 789 olarak üç parçaya ayrılır. Birinci parça aynı sırada kalır ve ikinci parça ters sıralanır. Daha sonra üçüncü parça aynı sırada alınır ve tablo boyutuna göre mod ile toplanır. (123+654+789 = 1566)

Hash Fonksiyonları

Division (Bölme)

- Anahtar değeri tablo boyutuna göre mod ile bölünür.
  - Örnek: SSN = 123456789 olarak verilsin. Tablo boyutu 1000 olursa, hash fonksiyonu sonucu aşağıdaki gibi bulur;

    hash(h) = 123456789 % 1000 = 789
- Yapısı basittir ancak çakışma olur.

Hash Fonksiyonları

- Mid-square (Orta kare)
  - Anahtar değerin karesi alınır ve sonucun orta kısmı seçilerek tablodaki pozisyon değeri bulunur.
    - Örnek: anahtar = 3121 olarak verilsin. Hash fonksiyonu sonucu aşağıdaki gibi bulur;

      $3121^2 = 9740641$

      hash(3121) = 406
  - Anahtarın karesi binary olarak gösterilebilir.

    $3121^2 = 10010100101000001011000001$

    hash(3121) = 0101000010 = 322

Hash Fonksiyonları

➢ Extraction (Seçme)

- Anahtar değerinin sadece bazı kısımları seçilerek tablodaki pozisyon değeri bulunur.
- Örnek: anahtar = 123-45-6789 olarak verilsin. Hash fonksiyonu aşağıdakilerden herhangi birisi olabilir;
  - hash(123-45-6789) = 123456789 = 1234
  - hash(123-45-6789) = 123456789 = 6789
  - hash(123-45-6789) = 123456789 = 1289

Hash Fonksiyonları

➢ Radix Transformation (Taban Dönüştürme)

- Anahtar değeri başka bir sayı tabanına dönüştürülür.
  - Örnek: anahtar = 1238 olarak verilsin. Tablo boyutu 1000 olarak alındığında,

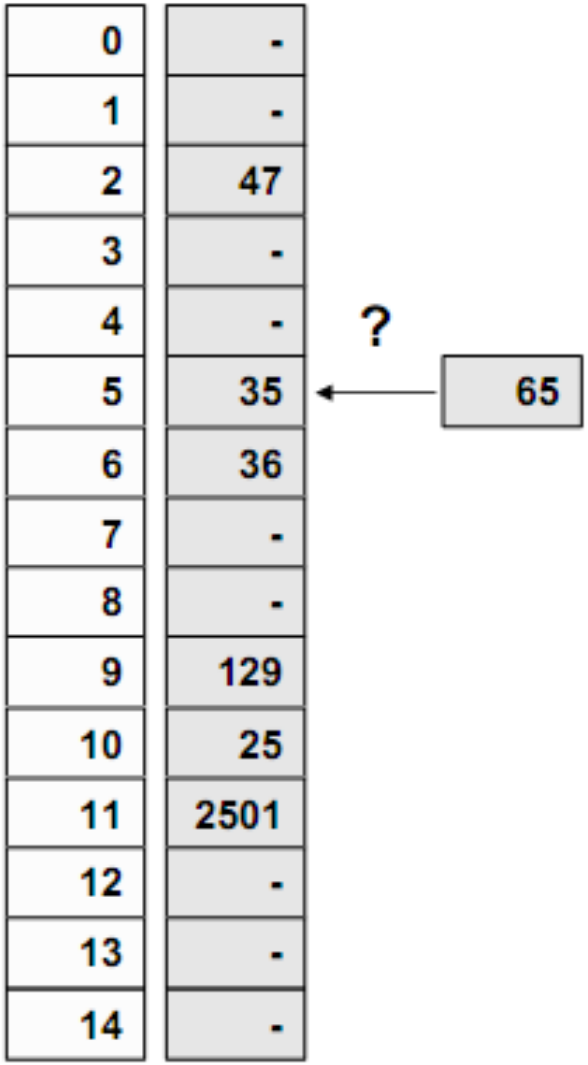
    $1238_{10} = 2326_8$
  - Hesaplanan değer tablo boyutuna mod ile bölünerek pozisyon değeri bulunur.

    Hash(1238) = 2326 % 1000 = 326

Çakışma (Collision)

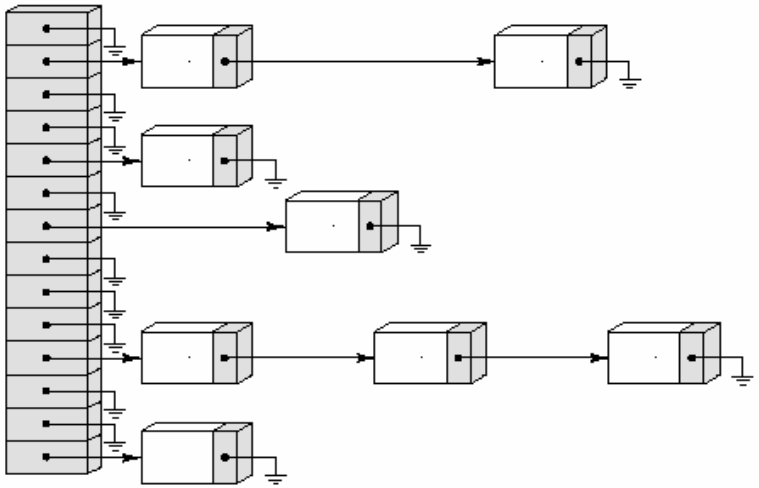
- Çakışma(Collision), hashing algoritmalarında, iki farklı anahtarın(key) aynı hash değerini üretmesi durumudur.
- Örneğin 65 değerini h(x) = x%15 e göre eklemeye çalıştığımızda aynı pozisyona daha önceden 35 değerinin eklendiğini görürüz. Aynı pozisyona birden fazla kayıt gelirse çakışma meydana gelir.

  - h(x) =65 mod 15 = 5
  - h(x) =35 mod 15 = 5



Çakışmanın Giderilmesi

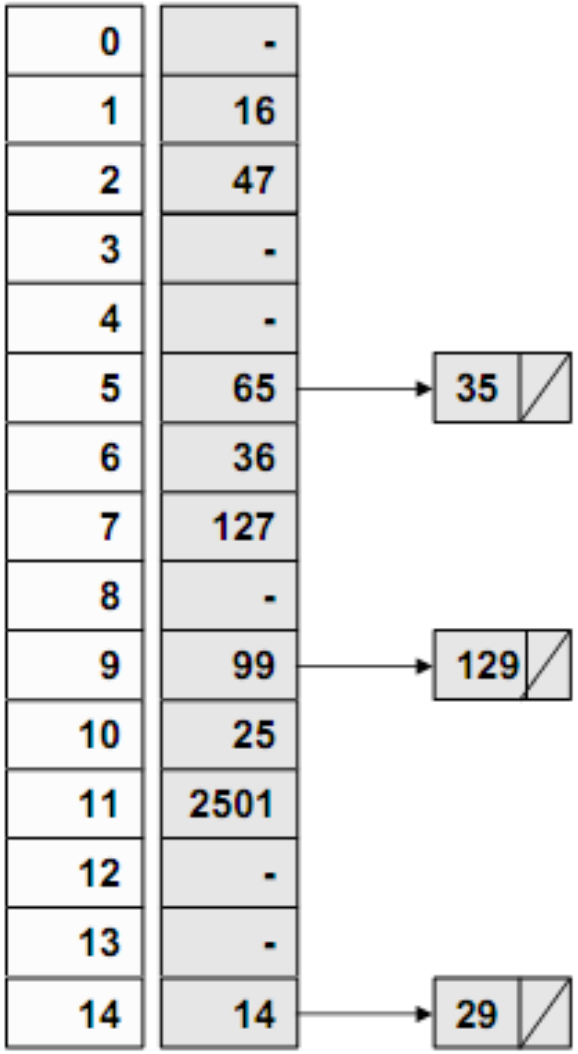
- Chaining(Zincirleme)
  - Her hash tablosu indeksine bağlı bir liste kullanılır.
  - Çakışan anahtarlar bu listenin içine eklenir.
  - Ekleme: Yeni eklenen listenin başına eklenir. 65 sayısı liste başına eklendi.
  - Silme/Erişim: Uygun listede arama yapılır.



0	-
1	-
2	47
3	-
4	-
5	65
6	36
7	-
8	-
9	129
10	25
11	2501
12	-
13	-
14	-

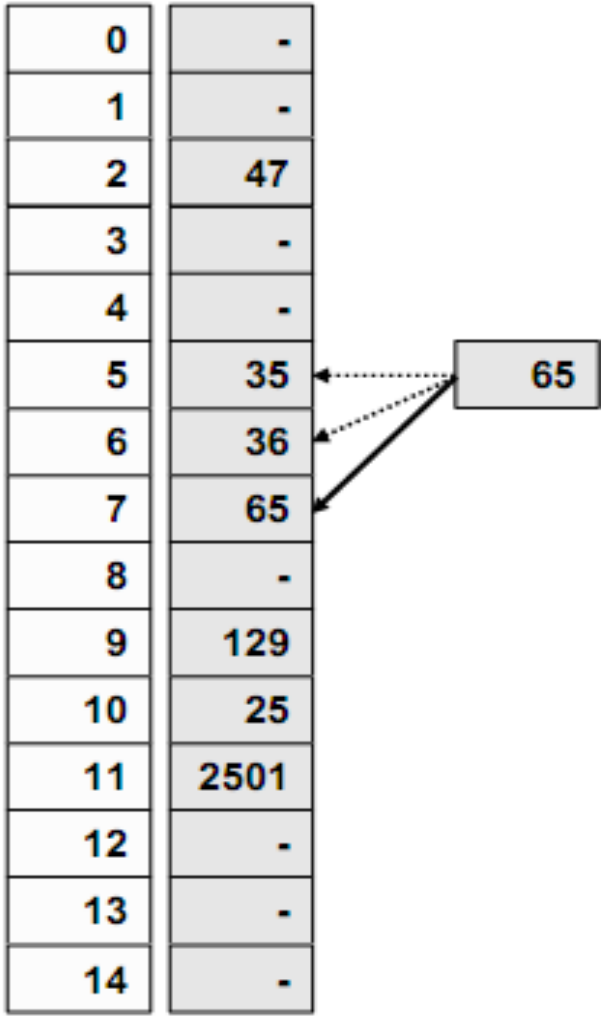
Çakışmanın Giderilmesi

- Chaining(Zincirleme)
  - Örnek:
    - 29, 16, 14, 99, 127 sayılarının eklenmesi
  - Aynı pozisyona gelen diğer anahtarlar bağlı listenin başına eklenmektedir.
  - Chaining metodunun dezavantajları
    - Tablonun bazı kısımları hiç kullanılmamaktadır.
    - Bağlı listeler uzadıkça arama ve silme işlemleri için gereken zaman uzamaktadır.



Çakışmanın Giderilmesi

- Linear Probing
  - İlk boş indekse yerleştirilir.
  - Ekleme: Boş bir alan bulunarak yapılır.
  - Silme/Erişim: İlk boş alan bulunana kadar devam edebilir.



Çakışmanın Giderilmesi

- Linear Probing
  - Örnek :
    - 18, 41, 22, 44, 59, 32, 31, 73 değerlerini h(x) = x mod 13 algoritmasına göre verilen sırada yerleştirme

0	1	2	3	4	5	6	7	8	9	10	11	12

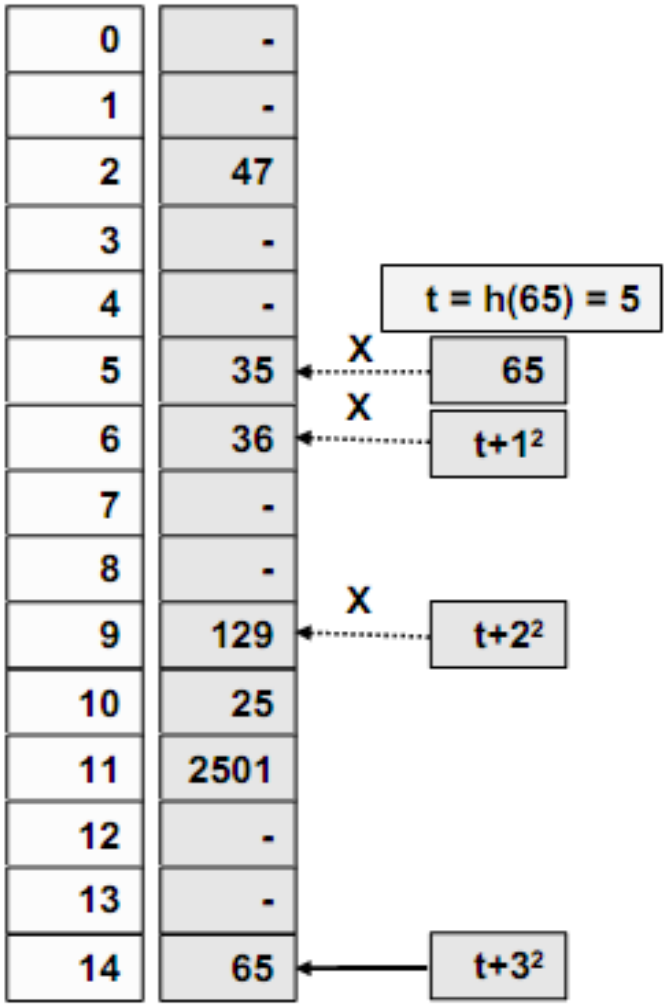
		41			18	44	59	32	22	31	73	
0	1	2	3	4	5	6	7	8	9	10	11	12

Çakışmanın Giderilmesi

- Linear Probing
  - Linear Probing metodunun avantajları / dezavantajları
    - Bağlı listeler gibi ayrı bir veri yapısına ihtiyaç duyulmaz.
    - Kayıtların yığın şeklinde toplanmasına sebep olur.
    - Silme ve arama işlemleri için gereken zaman aynı hash değeri sayısı arttıkça artar.

Çakışmanın Giderilmesi

- Quadratic Probing
  - Aynı pozisyona gelen ikinci kayıt ikinci dereceden fonksiyonla yerleştirilir. Daha geniş aralıklarla boş indeks aranır.
  - En çok kullanılan fonksiyon
    - **h(x) = x % m = t**
    - **f(x, i) = (h(x) + i²) % m**
    - ***x:anahtar, m: tablo boyutu***
  - Yeni pozisyon için sırasıyla ($t+1^2$), ($t+2^2$), ..., ($t+n^2$) değerlerine karşılık gelen pozisyonlara bakılır ve ilk boş olana yerleştirilir.



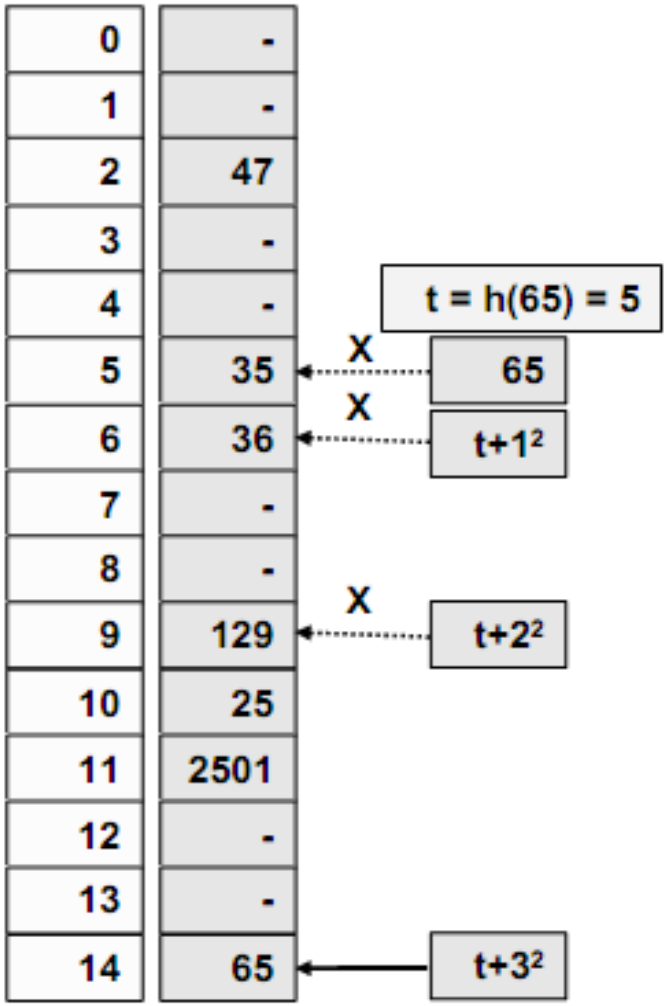
Çakışmanın Giderilmesi

- Quadratic Probing
  - Örnek:
    - 65 değerini hash tablosuna quadratic probing metoduyla yerleştirme

    h(x) = x % m = t

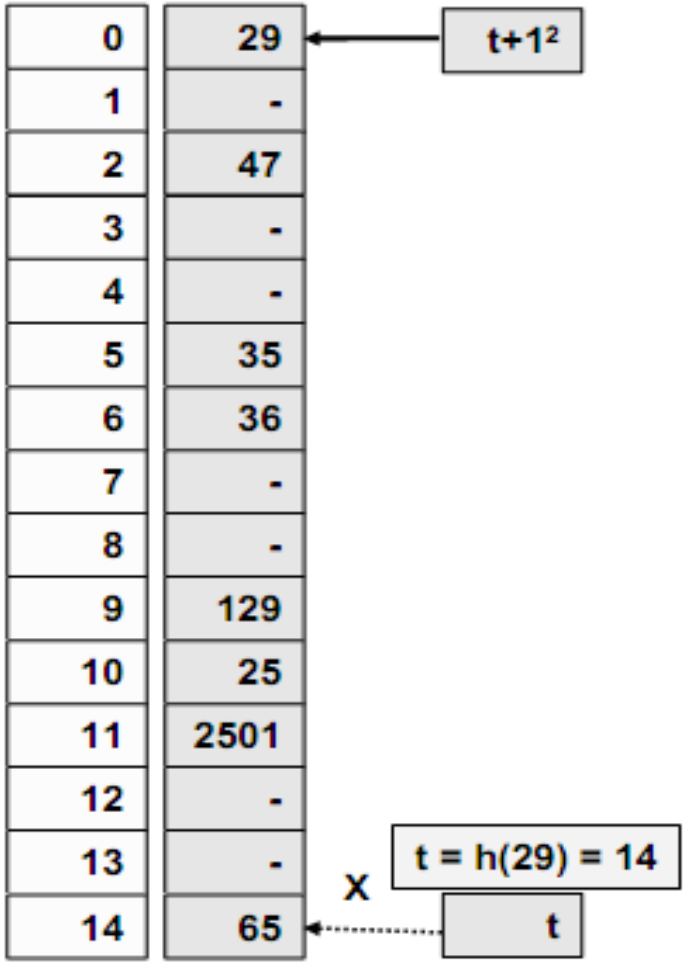
    f(x, i) = (h(x) + i²) % m

    x: anahtar, m: tablo boyutu, i: iter
  - h(65) = 65 % 15 = 5
  - f(65, 0) = (5+0²) % 15 = 5 (dolu)
  - f(65, 1) = (5+1²) % 15 = 6 (dolu)
  - f(65, 2) = (5+2²) % 15 = 9 (dolu)
  - f(65, 3) = (5+3²) % 15 = 14 (boş)



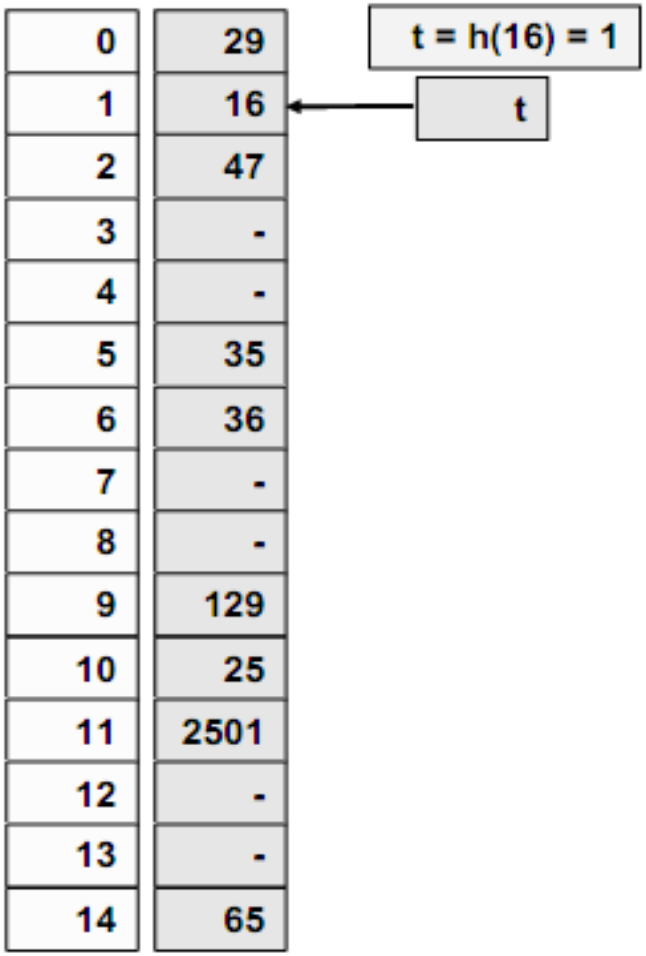
Çakışmanın Giderilmesi

- Quadratic Probing
  - Örnek:
    - 29, 16, 14, 99, 127 değerlerini hash tablosuna quadratic probing metoduyla sırasıyla yerleştirme
  - h(x) = x mod m
  - h(29) = 29 % 15=14 (dolu)
  - f(29, 1) = (14+1²) % 15 = 0 (boş)



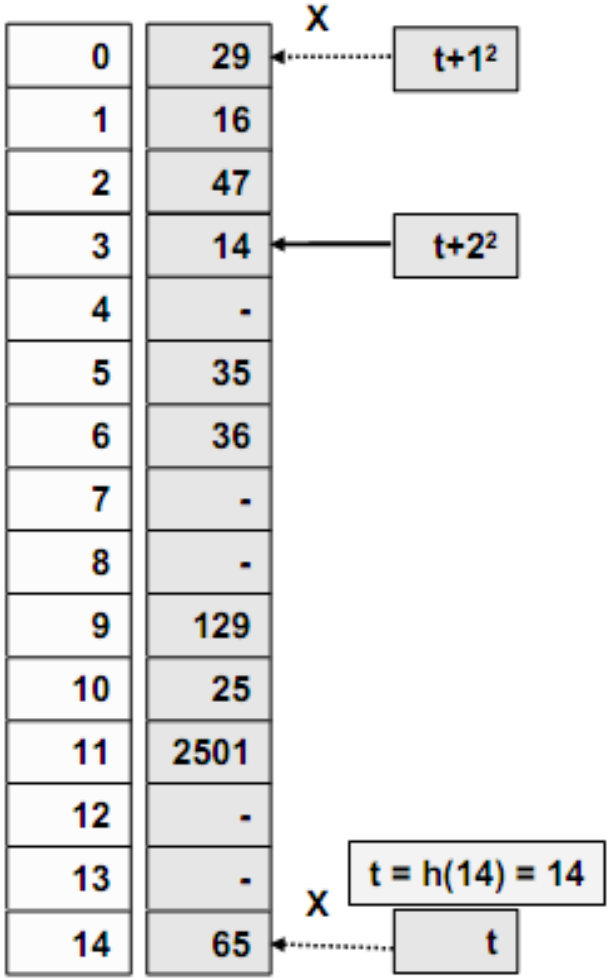
Çakışmanın Giderilmesi

- Quadratic Probing
  - Örnek:
    - 29, 16, 14, 99, 127 değerlerini hash tablosuna quadratic probing metoduyla sırasıyla yerleştirme
  - h(16) = 16 % 15=1



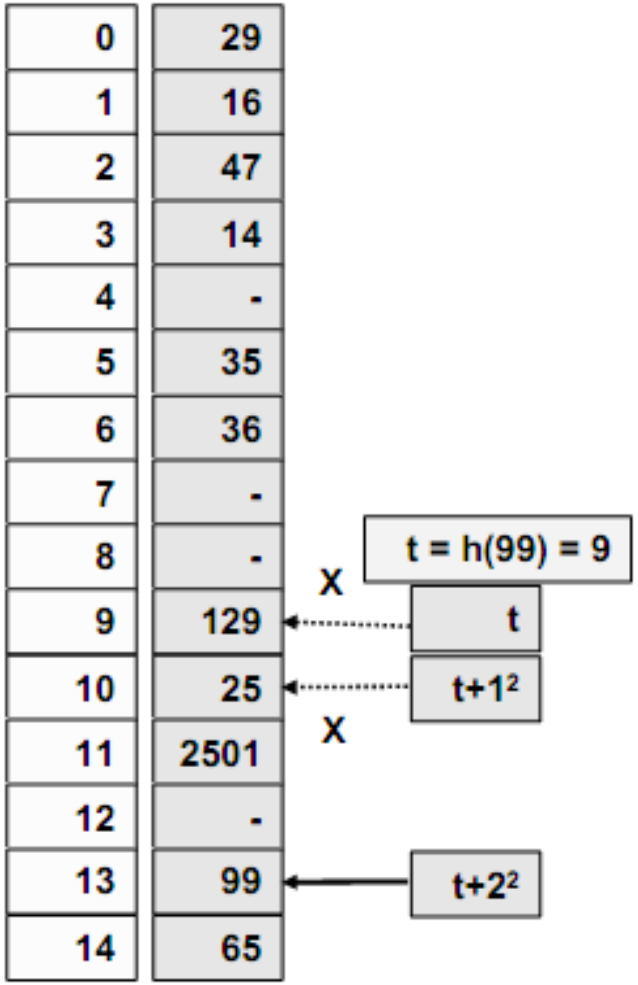
Çakışmanın Giderilmesi

- Quadratic Probing
  - Örnek:
    - 29, 16, 14, 99, 127 değerlerini hash tablosuna quadratic probing metoduyla sırasıyla yerleştirme
  - h(14) = 14 % 15=14
  - f(14, 1) = (14+1²) % 15 = 0
  - f(14, 2) = (14+2²) % 15 = 3



Çakışmanın Giderilmesi

- Quadratic Probing
  - Örnek:
    - 29, 16, 14, 99, 127 değerlerini hash tablosuna quadratic probing metoduyla sırasıyla yerleştirme
  - h(99) = 99 % 15=9
  - f(99,1)=(9+1²)%15=10
  - f(99,2)=(9+2²)%15=13%15=13



Çakışmanın Giderilmesi

- Quadratic Probing
  - Örnek:
    - 29, 16, 14, 99, 127 değerlerini hash tablosuna quadratic probing metoduyla sırasıyla yerleştirme
  - h(127) = 127 % 15= 7

0	29
1	16
2	47
3	14
4	-
5	35
6	36
7	127
8	-
9	129
10	25
11	2501
12	-
13	99
14	65

t = h(127) = 7

t

Çakışmanın Giderilmesi

- Quadratic Probing
  - Quadratic Probing metodunun avantajları / dezavantajları
    - Anahtar değerlerini linear probing metoduna göre daha düzgün dağıtır.
    - Yeni eleman eklemede tablo boyutuna dikkat edilmezse sonsuza kadar çalışma riski vardır. Örneğin; boyutu 16 (0-15) olan bir tabloda 0, 1, 4 ve 9 pozisyonlarının dolu olduğu durumda 16 değerini eklemeye çalıştığımız zaman sonsuz döngüye girer.

Çakışmanın Giderilmesi

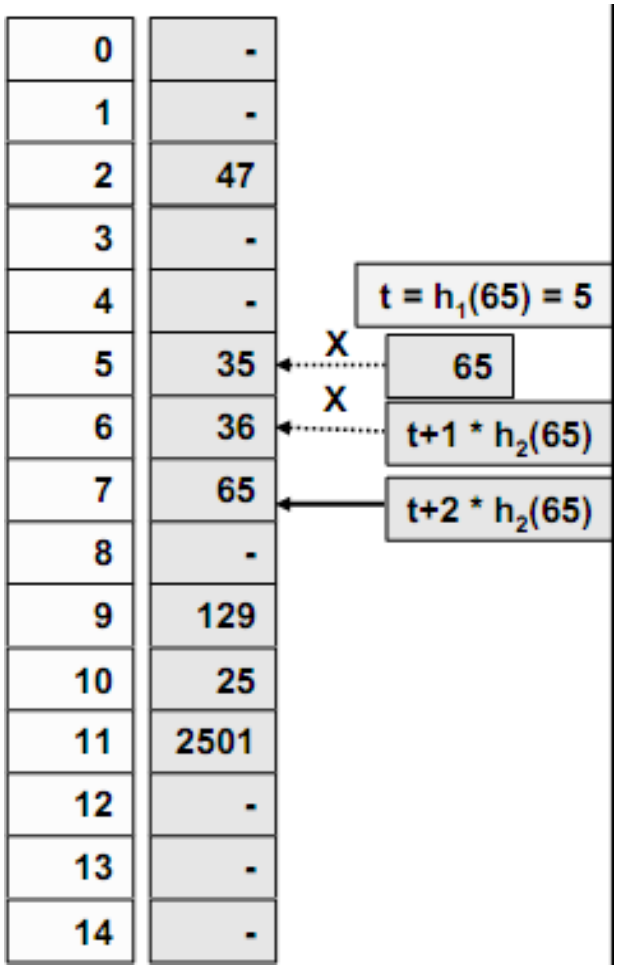
- Double Hashing
  - Aynı pozisyona gelen ikinci kayıt için ikinci bir hash fonksiyonu kullanılır.
  - İkinci hash fonksiyonu 0 değerini alamaz.
  - En çok kullanılan fonksiyon:

    hash(x, i) = (hash1(x) + i * hash2(x)) % m

    hash2(x) = R − (x % R) R < m, R: asal (veya m ile aralarında asal)
  - Pozisyon şu şekilde bulunur:
    - hash(x) = hash1(x)
    - hash(x) = (hash1(x) + 1 * hash2(x)) % m
    - hash(x) = (hash1(x) + 2 * hash2(x)) % m
    - hash(x) = (hash1(x) + 3 * hash2(x)) % m …

Çakışmanın Giderilmesi

- Double Hashing
  - Örnek: 65 değerinin eklenmesi
  - hash1(65)= 65 % 15 = 5
  - hash2(65)=11 − (65 % 11) = 1
  - hash(65,0) = (hash1(65) + 0*hash2(65)) %15 = 5 dolu
  - hash(65,1)=(5 + 1*hash2(65))%15
    =(5 + 1*1)%15 = 6 (dolu)
  - hash(65,2) = (5 + 2*1)%15 = 7 (boş)

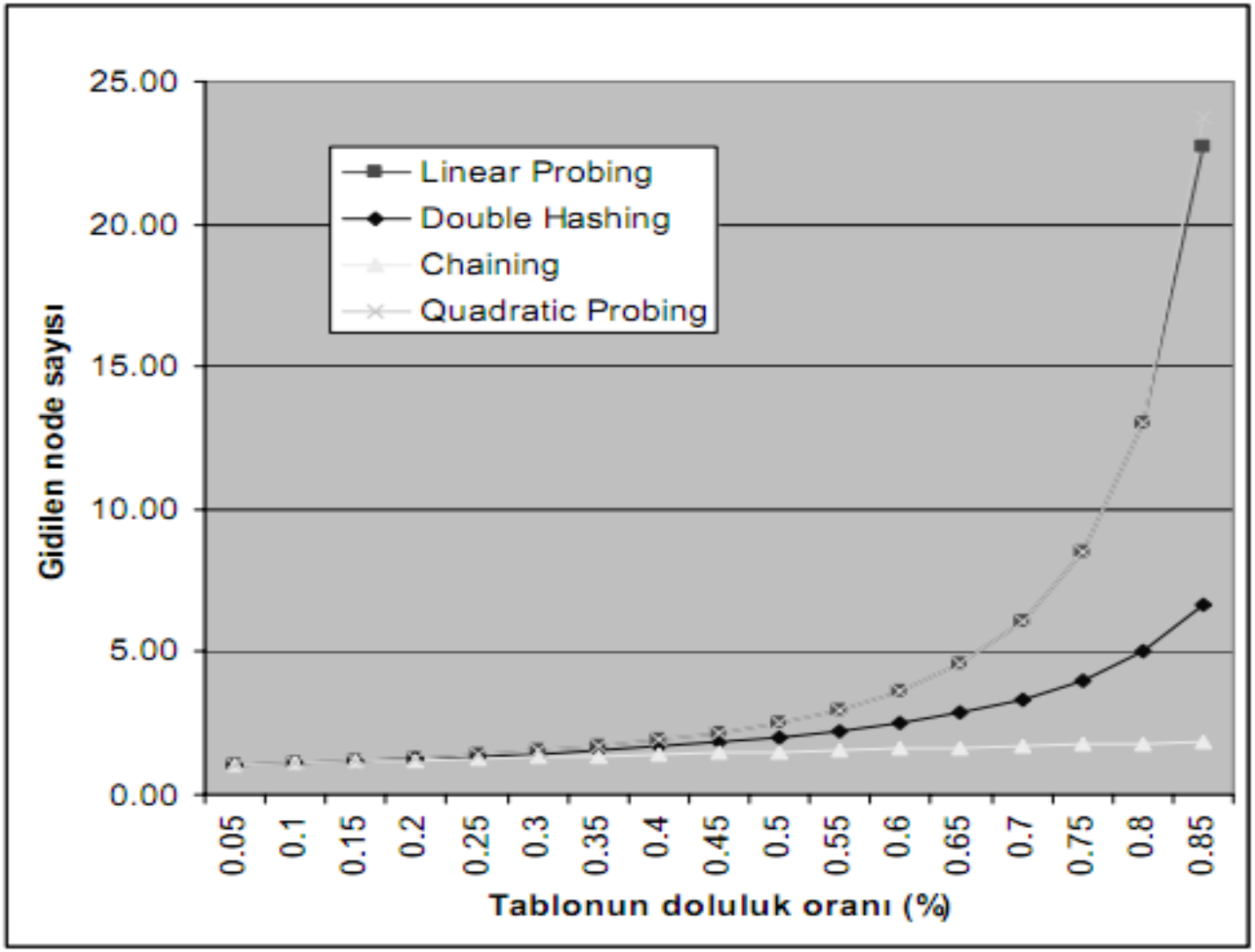


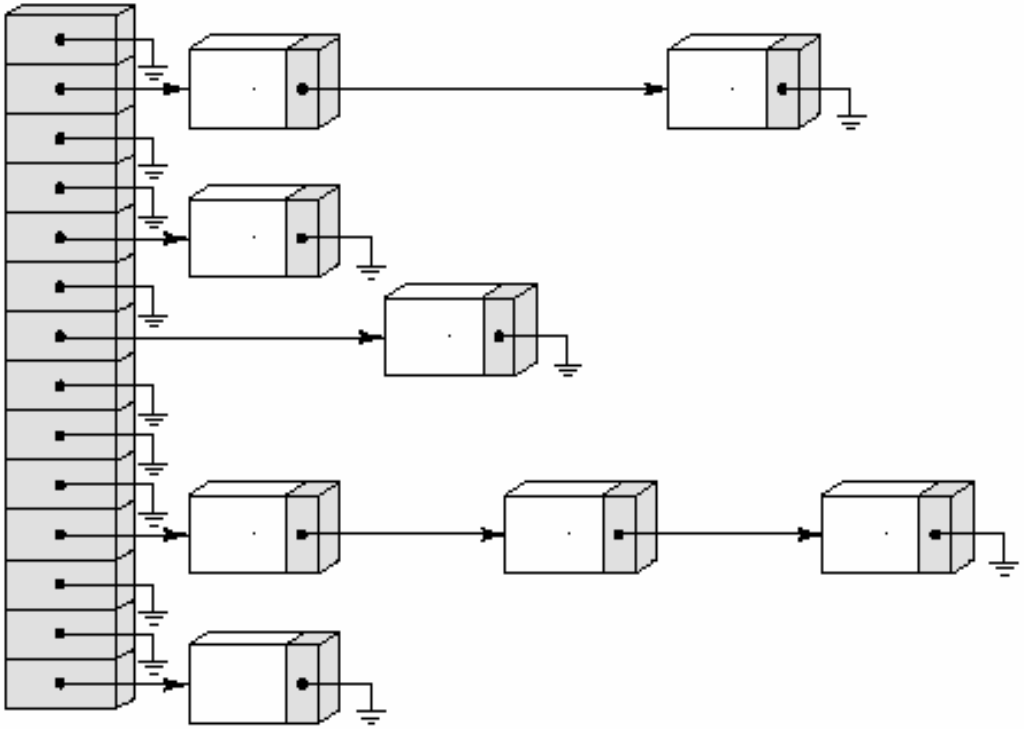
Çakışmanın Giderilmesi

- Double Hashing
  - Double Hashing metodunun avantajları / dezavantajları
    - Anahtar değerlerini linear probing metoduna göre daha düzgün dağıtır ve gruplar oluşmaz.
    - Quadratic probing metoduna göre daha yavaştır çünkü ikinci bir hash fonksiyonu hesaplanır.

Çakışmanın Giderilmesi

- Performans



Örnek Uygulamalar

1) Klavyeden girilen integer tipinde anahtar (key) değerine karşılık string tipinde value değerlerini tutan bir Hash Table oluşturunuz. Çakışmaları Bağlantılı liste kullanarak Chaining yolu ile düzelten kodu yazınız.